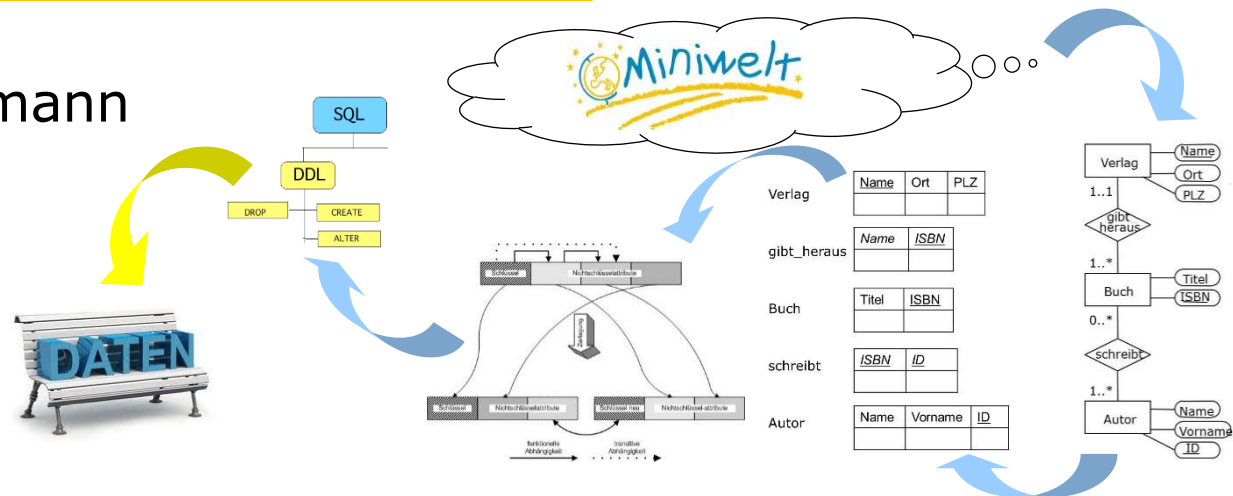


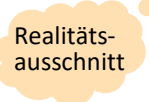
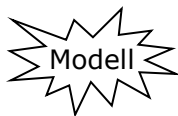
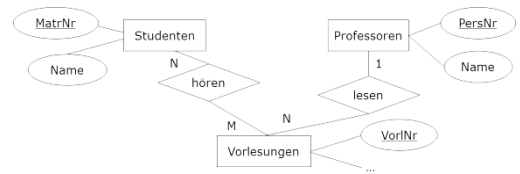
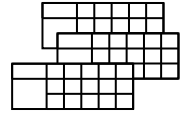
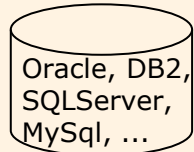
SQL-Datendefinitionssprache

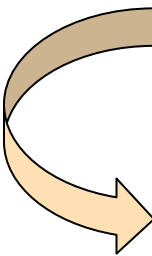
Josef Altmann



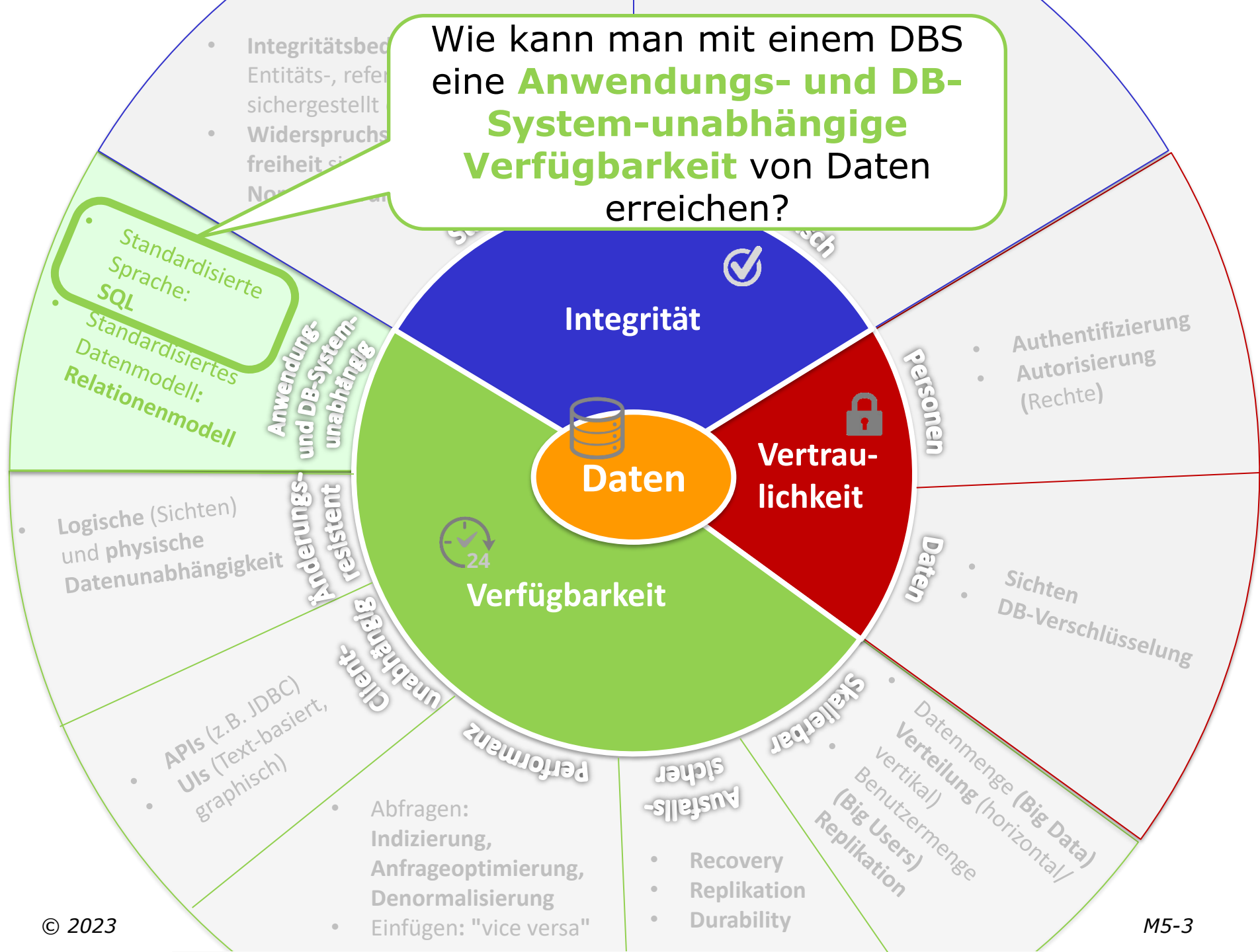
Datenbankentwurfsschritte

Umsetzung logisches in physisches Schema

Phase	Modell	Ergebnis
Anforderungsanalyse	Requirements Engineering	Pflichtenheft (Use-Cases, ...)  Realitätsausschnitt ANF 5.4: Eine Vorlesung wird durch eine eindeutige Vorlesungsnummer identifiziert. ...
Konzeptueller Entwurf	Entity Relationship	Konzeptuelles DB-Schema  Modell 
Logischer Entwurf	Datenbankmodell	Logisches DB-Schema  Vorlesung{ <u>VorlNr</u> , Titel, ...} Studenten{ <u>MatrNr</u> , Name, ...}
Physischer Entwurf	Data Definition Language	Physisches DB-Schema  Oracle, DB2, SQLServer, MySQL, ... <pre>CREATE TABLE Vorlesung (VorlNr NUMBER(10) PRIMARY KEY, Titel VARCHAR(20), ...);</pre>



Wie kann man mit einem DBS eine **Anwendungs- und DB-System-unabhängige Verfügbarkeit** von Daten erreichen?



Überblick

- Einführung Datendefinitionssprachen
- Datendefinition in SQL
 - Schemata
 - Datentypen
 - Attribute und Tabellen
 - Referentielle Integrität
 - Schemaevolution
- Anhang I: Universitätsschema
- Anhang II: Definitionen
- Anhang III: Exkurs **CHECK**-Klausel

Structured Query Language

Standardisierung

- SQL ist die in der Praxis am **weitesten verbreitete Datenbanksprache** für relationale Datenbanken
- Die **Historie** von SQL geht zurück bis **1974**, die Anfangszeit der Entwicklung relationaler Datenbanken
- Der Sprachumfang von SQL ist einer **permanenten Weiterentwicklung** und Standardisierung unterworfen. Derzeit relevant sind der Stand von **1992**, **1999** und **2003** entsprechend bezeichnet mit SQL-92, SQL:1999 und SQL:2003
- Auf dem **Markt befindliche Datenbanksysteme** realisieren weitgehend die im Standard **SQL-92** definierten Konzepte und bereits viele Konzepte der Standards **SQL:1999**, **SQL:2003**, **SQL:2008**, **SQL:2011** und **SQL:2016**
- Aktueller Standard ist **SQL:2023**
www.iso.org/standard/76583.html

Structured Query Language

Standardisierung

- **SQL ist "de facto"-Standard** in der **relationalen Welt**
 - 1986 von ANSI (SQL1), 1987 von ISO akzeptiert
- Weiterentwicklung des Standards in mehreren Stufen:
 - **SQL2 (SQL-92)**
 - erster „kompletter“ Standard
 - **(SQL3) SQL:1999**
 - viele neue Anforderungen
 - Standard nicht umfangreich umgesetzt (DBMS „hinken“ hinterher)
 - **SQL:2003**
 - XML-Integration
 - Funktionsexplosion!
 - **SQL:2008**
 - Umfangreiche Entwicklungen bei SQL/XML
 - **SQL:2016**
 - Umfangreiche Entwicklungen bei SQL/JSON
 - **SQL:2023**
 - aktuelle Version des Standards

SQL-Standard Entwicklungshistorie

Jahr	Entwicklung
1975	<i>SEQUEL</i> = <i>Structured English Query Language</i> , der Vorläufer von <i>SQL</i> , wird für das Projekt System R von IBM entwickelt
1979	<i>SQL</i> gelangt mit <i>Oracle V2</i> erstmals durch <i>Relational Software Inc.</i> auf den Markt
1986	SQL1 wird von ANSI als Standard verabschiedet
1987	SQL1 wird von der Internationalen Organisation für Normung (ISO) als Standard verabschiedet und 1989 überarbeitet – Hinzufügung von Integritätsbedingungen
1992	Der Standard SQL2 oder SQL-92 wird von der ISO verabschiedet
1999	SQL3 oder SQL:1999 wird verabschiedet. Features wie Trigger , rekursive Abfragen, reguläre Ausdrücke und OO-Erweiterungen werden hinzugefügt
2003	SQL:2003 fügt erste Änderungen für besseren XML-Support ein und führt die OVER -Klausel zur Unterstützung von Data Warehousing ein
2006	SQL/XML:2006 legt genauer fest, wie <i>SQL</i> in Zusammenhang mit <i>XML</i> verwendet werden kann
2008	SQL:2008 fügt „ INSTEAD OF “- Trigger und das TRUNCATE - und FETCH FIRST -Statement hinzu
2011	SQL:2011 erweitert die OVER -Klausel und erweitert die analytischen Funktionen
2016	SQL:2016 ist die aktuelle Revision des Standards und führt LIST_AGG -Statement und JSON -Unterstützung ein.
2023	SQL/PGQ unterstützt Property Graph Queries . SQL/JSON -Typ mit Funktionen.

Lebenszyklus ISO-Standardisierung



Gründe für unterschiedliche Umsetzung des SQL-Standards durch Hersteller

- Es gibt verschiedenste **Gründe**, warum die DBS-Hersteller, den Standard nur teilweise umsetzen:
 - **Komplexität** und **Größe** des **SQL-Standards** machen eine komplette Umsetzung schwierig bis unmöglich
 - Standard bestimmt teilweise das **konkrete DB-Verhalten nicht** und bietet damit Spielraum für unterschiedliche Umsetzungen
 - Neuere Versionen des Standards **brechen** teilweise die **Rückwärtskompatibilität** → Hersteller möchten dies ihren Kunden nicht zumuten
 - **Vendor-Lock-In**: Hersteller haben wenig Interesse, dass Produkte austauschbar sind
 - Benutzer **favorisieren Performance** über **Standard-Compliance**
 - ...

Structured Query Language

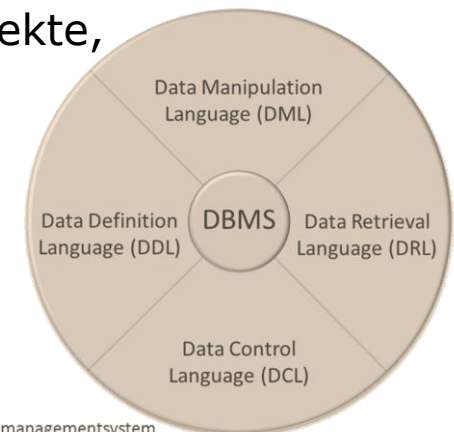
■ SQL stellt eine standardisierte

- **Datendefinitionssprache** (DDL)
 - Erstellen, Ändern und Löschen von Datenbankobjekten (Tabellen, etc.)
- **Datenmanipulationssprache** (DML)
 - Einfügen, Ändern und Löschen von Daten
- **Anfrage (Query)-Sprache** (DRL)
 - Abfragen von Daten (dies ist der komplexeste Teil von SQL)
- **Kontrollsprache** (DCL)
 - Verwalten von Zugriffsrechten auf Datenbankobjekte, Transaktionskontrolle, etc.

bereit.

■ SQL ist

- **deklarativ**
- **mengenorientiert**



DBMS =
Datenbankmanagementsystem

Überblick

- Einführung Datendefinitionssprachen
- Datendefinition in SQL
 - Schemata
 - Datentypen
 - Attribute und Tabellen
 - Referentielle Integrität
 - Schemaevolution
- Anhang I: Universitätsschema
- Anhang II: Definitionen

Structured Query Language

Datendefinition

- **SQL-DDL** umfasst **Klauseln (Anweisungen)**, mit denen
 - Schemata
 - Datentypen
 - Attribute
 - Basistabellen
 - Integritätsbedingungen (Schlüsselbedingungen, Wertebereichsbedingungen etc.)
 - Sichten
 - Indizes
 - ...definiert werden können

- **Datenbankschema** umfasst alle **Informationen über** die **Datenbank** (d.h. alles außer den eigentlichen Daten)

Datenbank-Realisierung

Basistabelle

Logisches
Datenbankschema
(Relationenmodell)

Student

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2



Realisierung des
Relationenmodells mit Hilfe
von SQL-DDL-Anweisungen

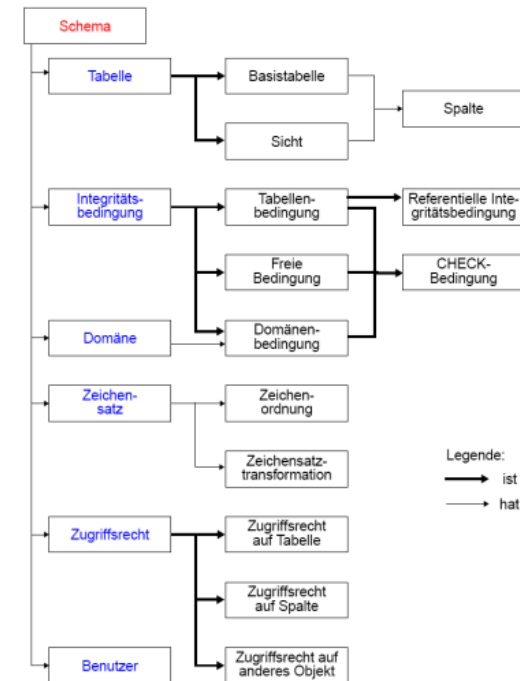
Physisches
Datenbank-
schema

```
CREATE TABLE Student (
  MatrNr      INTEGER PRIMARY KEY,
  Name        VARCHAR(30) NOT NULL,
  Semester    INTEGER DEFAULT 1,
  CONSTRAINT Semesterpruefung
  CHECK (Semester BETWEEN 1 AND 6) );
```

Schemadefinition

- Logisch zusammengehörige Schemaobjekte (Basistabellen, Sichten, Zugriffsrechte etc.) werden unter einem **gemeinsamen Schema** (~ Namensraum) verwaltet
- Jedes Schema ist einem **Benutzer zugeordnet**, z.B. DBA
- Schemaname = **Benutzername**, falls keine explizite Namensangabe erfolgt
- Schemata werden im **Data Dictionary** gespeichert

Elemente des SQL-Schemas



```
CREATE SCHEMA [schema] [AUTHORIZATION user]
...
[schema-element-list]
```

Beispiel

```
CREATE SCHEMA BeispielDB AUTHORIZATION altmann
```

Datentypen

- **Spalte** kann nur **Werte** eines bestimmten **Datentyps** aufnehmen
- **DBMS** bieten **vordefinierte Datentypen** an
 - Relativ **standardisiert**
 - Zeichenketten (feste, variable Länge)
 - Zahlen (Integer, Fest- und Gleitkommazahl)
 - Unterstützt, aber in jedem **DBMS verschieden**
 - Lange Zeichenketten (CLOB)
 - Binäre Daten (BLOB)
 - Datums- und Zeitwerte
 - Intervalle, Zeitstempel
 - XML
 - JSON
 - ...

Datentypen

SQL: Basisdatentypen

BOOLEAN	Wahrheitswert	TRUE, FALSE, UNKNOWN
INTEGER SMALLINT BIGINT	ganze Zahl	1704
DECIMAL (p, q) NUMERIC (p, q)	Festkommazahl mit q Nachkommastellen	1003.44
FLOAT (p) DOUBLE	Fließkommazahl	1.5E-4
CHAR (q) VARCHAR (q) CLOB	Zeichenkette mit fester Länge q variable Länge bis max. q lange Zeichenketten	'SQL:1999' Textdateien
BIT (q) BLOB	binäre lange Bitfolgen	B '11011011' Multi-Media-Objekte
DATE TIME TIMESTAMP INTERVAL	Datum Zeit Zeitstempel Zeitintervall	DATE '1999-06-19' TIME '11:30:49' TIMESTAMP '2002-08-23 14:15:00' INTERVAL '48' HOUR
XML	XML-Wert	<Titel>SQL:2003</Titel>
JSON	JSON-Wert	{ "Titel": "SQL:2003" }

Datentypen

Oracle19c - Basisdatentypen

- **VARCHAR2(size)** Zeichendaten variabler Länge (Für size muss eine Maximalgröße angegeben werden: Mindestgröße 1, Maximalgröße 4.000)
- **CHAR[(size)]** Zeichendaten fester Länge mit der Größe size (Default- und Mindestgröße 1 Byte, Maximalgröße 2.000 Byte)
- **NUMBER[(precision [, scale])]** Wert mit der Gesamtstellenzahl precision und der Anzahl der Nachkommastellen scale (Gesamtstellenzahl 38)
 - **ANSI-Datentypen** (**INTEGER**, **DECIMAL**, **FLOAT**, **CHARACTER**) sind von **NUMBER**-Datentyp abgeleitet
- **DATE** Datums- und Uhrzeitwerte bis auf die nächste Sekunde vom 1. Januar 4712 v. Chr. bis zum 31. Dezember 9999 n. Chr.
- **CLOB** Zeichendaten (zwischen 16 und 128 TB)
- **BLOB** Binärdaten (zwischen 16 und 128 TB)
- **BFILE** Referenz auf eine Datei/Ordner außerhalb der Datenbank
- **ROWID** Zeiger auf eine Zeile in einer Tabelle
- **XMLType** Für XML-Dokumente
- **JSON** Für JSON-Dokument (bis Oracle19c VARCHAR2/BLOB, ab 21c eigener JSON-Type)
- ...

Definition von Attributen

■ Spezifikation von

- Attributname mit **Typfestlegung**
- Integritätsbedingungen $\Rightarrow$ **CONSTRAINTS**
 - stellen die Korrektheit der Daten sicher
 - werden aus der modellierten Miniwelt an DBS delegiert
 - weisen Änderungen zurück, die die Integrität verletzen

```
column-def
 ::= column {data-type}
    [ DEFAULT {literal|NULL}
    [column-constraint-def-list]
```

Beispiele

```
Name  VARCHAR(30)           -- Attributname und Datentyp,
Rang   CHAR(2) DEFAULT 'C3' -- Default-Wert und
RaumNr INTEGER PRIMARY KEY -- Integritätsbedingung
```

Definition von Integritätsbedingungen

CONSTRAINTS

■ Integritätsbedingungen $\Rightarrow$ CONSTRAINTS

- stellen die **Korrektheit** der **Daten** sicher
- werden aus der **modellierten Miniwelt** abgeleitet, an DBS delegiert und gelten anwendungsweit
- **zentral** im **DBS** definiert (Data Dictionary)
- werden von **DBS überwacht**, ob **prüfungsrelevante Ereignisse eintreten** (INSERT, UPDATE, DELETE)
- **weisen Änderungen zurück**, die die Integrität verletzen

CONSTRAINT mit CHECK-Bedingung



Semantische Integrität

CONSTRAINT mit PRIMARY KEY



Entitätsintegrität

CONSTRAINT mit FOREIGN KEY und REFERENCES



Referentielle Integrität

Beispiele

Professoren haben Rang C2, C3, C4

-- Semantische Integrität

Matrikelnummer der Studierenden ist eindeutig

-- Entitätsintegrität

Anmeldung zur Übung nur bei aufrechter Inskription

-- Referentielle Integrität

Definition von Attributen 1/4

Integritätsbedingungen - CONSTRAINTS

- **NOT NULL**
Nullwerte verboten
- **CHECK (cond-exp)**
Attributspezifische Bedingung (Werte von Attributen einschränken)
- **UNIQUE (attribute-list)**
Jeder Wert darf nur einmal auftreten
- **PRIMARY KEY (attribute-list)**
Angabe eines Primärschlüssel
- **FOREIGN KEY (attribute-list)**
REFERENCES tabelle (attribute-list)
Angabe der referentiellen Integrität

Definition von Attributen 2/4

Integritätsbedingungen – NOT NULL

- **NOT NULL** schließt in Spalten Nullwerte als Attributwerte aus
- Kennzeichnung von Nullwerte in SQL durch **NULL**
- **NULL** repräsentiert die Bedeutung
 - „Wert unbekannt“,
 - „Wert nicht anwendbar“ oder
 - „Wert existiert nicht“,
 - gehört zu keinem Wertebereich
- **NULL** kann in allen Spalten auftreten, außer in Schlüsselattributen und den mit **NOT NULL** gekennzeichneten

Beispiel

```
Name VARCHAR(30) NOT NULL
```

Definition von Attributen 3/4

Details siehe Anhang III ➤

Integritätsbedingungen - CHECK

■ CHECK-Klausel

- Überprüfung allgemeiner statischer Integritätsbedingungen mittels **CHECK**-Klausel, gefolgt von einer Bedingung
- Auswertung der **CHECK**-Klausel bei Änderung/Einfügung in die Tabelle
- Änderungen auf einer Tabelle werden nur zugelassen, wenn **CHECK** zu `true` auswertet.
- Bedingungen können beliebig komplex sein, auch eine Anfrage enthalten
 - Achtung: nur sehr rudimentär implementiert!

Beispiele: Spaltenbedingungen

```
Semester INTEGER CHECK(Semester BETWEEN 1 AND 6)
Note NUMERIC(2,1) CHECK(Note BETWEEN 1.0 AND 5.0)
Rang CHAR(2) CHECK (Rang IN ('C2', 'C3', 'C4'))
```

Definition von Attributen 4/4

Benannte Integritätsbedingungen

- Vorteile der Vergabe von **Constraint-Namen** für die Integritätsbedingungen
 - Diagnosehilfe bei Fehlern
 - Aktivieren/Deaktivieren von benannten Integritätsbedingungen

```
column-constraint-def ::= [CONSTRAINT constraint]
                        {NOT NULL | {PRIMARY KEY | UNIQUE}
                        | reference-def | CHECK(cond-exp) }
```

Beispiel

```
Semester  INTEGER
          CONSTRAINT Semesterpruefung CHECK(Semester BETWEEN 1 AND 6)
```

Ausgabe ohne benutzerdef. Constraint-Namen am Beispiel von Oracle:  Name wird von DBMS generiert!
 SQL Error: ORA-02290: CHECK-Constraint (ALTMANN.**SYS_C007925**) verletzt

Ausgabe mit benutzerdef. Constraint-Namen am Beispiel von Oracle:
 SQL Error: ORA-02290: CHECK-Constraint (ALTMANN.**CHECK_Semesterpruefung**) verletzt

 Benutzerdefinierter Name!

Definition von Basistabellen

■ Spezifikation von

- Zugehörigen Attributen mit Typfestlegung
- Integritätsbedingungen (**CONSTRAINTS**)

```
CREATE TABLE base-table (base-table-element-list)
base-table-element
::= column-def | base-table-constraint-def
```

Beispiel

```
CREATE TABLE Student (
  MatrNr      INTEGER,
  Name        VARCHAR(30) CONSTRAINT Student_Name_NN NOT NULL,
  Semester    INTEGER DEFAULT 1,
              CONSTRAINT Semesterpruefung CHECK(Semester BETWEEN 1 AND 6),
              CONSTRAINT Student_PK PRIMARY KEY(MatrNr));
```

■ Wirkung des **CREATE TABLE**-Kommandos ist sowohl

- die Ablage des Relationenschemas im Data Dictionary, als auch
- die Vorbereitung einer "leeren Basistabelle" in der Datenbank

Referentielle Integrität

- **Schlüssel** identifizieren ein Tupel einer Relation.
- **Fremdschlüssel** verweisen auf Tupel einer in Beziehung stehenden Relation.

Beispiel

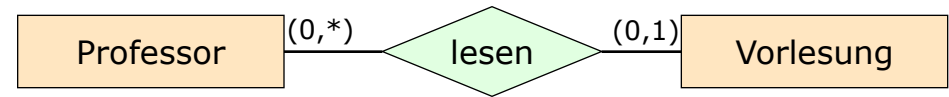
`PersNr` ist Schlüssel von `Professor`, `gelesenVon` in `Vorlesung` verweist auf Tupel in `Professor` und ist somit Fremdschlüssel. Es muss sichergestellt werden, dass in `gelesenVon` keine Werte stehen, die in `PersNr` nicht vorkommen.

Definition (referentielle Integrität)

Fremdschlüssel müssen entweder auf existierende Tupel einer anderen Relation verweisen oder einen Nullwert enthalten.

Referentielle Integritätsbedingung

zwischen Tabellen



- Referentielle Integritätsbedingungen entstehen zwischen **Primär-** und **Fremdschlüssel**.

Beispiel

```
CREATE TABLE Professor(
  PersNr    INTEGER PRIMARY KEY,
  Name      VARCHAR(30) NOT NULL,
  ...);
```

Vatertabelle

```
CREATE TABLE Vorlesung(
  VorlNr    INTEGER PRIMARY KEY,
  Titel      VARCHAR(30),
  SWS       INTEGER,
  gelesenVon INTEGER,
  CONSTRAINT FK_gelesenVon
    FOREIGN KEY (gelesenVon)
    REFERENCES Professor(PersNr)
);
```

Kindtabelle

- Zu jedem **Fremdschlüsselwert** der referenzierenden Kindtabelle (Vorlesung) muss ein entsprechender **Schlüsselwert** in der referenzierten Vatertabelle (Professor) existieren.



Frage?

- Welche Operationen auf der **Kindtabelle** gefährden potentiell die referentielle Integrität?

Beispiel

```
CREATE TABLE Professor(  
  PersNr    INTEGER PRIMARY KEY,  
  Name      VARCHAR(30) NOT NULL,  
  ...);
```

Vatertabelle

```
CREATE TABLE Vorlesung(  
  VorlNr    INTEGER PRIMARY KEY,  
  Titel      VARCHAR(30),  
  SWS       INTEGER,  
  gelesenVon INTEGER,  
  CONSTRAINT FK_gelesenVon  
    FOREIGN KEY (gelesenVon)  
    REFERENCES Professor(PersNr)  
);
```

Kindtabelle

Gewährleistung der referentiellen Integrität

Potentielle Gefährdung der Kindtabelle

- Operationen in der Kindtabelle
 - Einfügen eines Tupels
 - Ändern des FS in einem Tupel
 - Löschen eines Tupels
- Welche Maßnahmen sind erforderlich?
 - Beim Einfügen erfolgt eine Prüfung, ob in einem Vater-Tupel ein PS/SK-Wert gleich dem FS-Wert des einzufügenden Tupels existiert
 - Beim Ändern eines FS-Wertes erfolgt eine analoge Prüfung
 - Löschen eines Tupels ist immer unkritisch!



Frage?

- Welche Operationen auf der **Vatertabelle** gefährden potentiell die referentielle Integrität?

Beispiel

```
CREATE TABLE Professor(  
  PersNr    INTEGER PRIMARY KEY,  
  Name      VARCHAR(30) NOT NULL,  
  ...);
```

Vatertabelle

```
CREATE TABLE Vorlesung(  
  VorlNr    INTEGER PRIMARY KEY,  
  Titel      VARCHAR(30),  
  SWS       INTEGER,  
  gelesenVon INTEGER,  
  CONSTRAINT FK_gelesenVon  
    FOREIGN KEY (gelesenVon)  
    REFERENCES Professor(PersNr)  
);
```

Kindtabelle

Gewährleistung der referentiellen Integrität

Potentielle Gefährdung der Vaternabelle

■ Operationen in der Vaternabelle

- Löschen eines Tupels
- Ändern des PS/SK in einem Tupel
- Einfügen eines Tupels

Vorlesung

...	gelesenVon
...	2137
...	2125
...	2126
...	...

Professor

PersNr	...
2137	...
2125	...
2136	...
...	...

■ Welche Aktion ist wann möglich/sinnvoll?

- Verbiete Operation
- Lösche/ändere rekursiv Tupel mit zugehörigen FS-Werten
 - Falls Kind-Tupel erhalten bleiben soll (nicht immer möglich, z.B. bei Existenzabhängigkeit), setze FS-Wert zu NULL oder Default
- Einfügen eines Tuples ist immer unkritisch!

➤ **Referentielle Aktionen** nur für das Löschen und Ändern von Vater-Tupeln von Interesse!

Referentielle Aktionen

ON DELETE und ON UPDATE

- Detailliertere Spezifikation der **referentiellen Aktionen** für jeden Fremdschlüssel
 - Ist NULL verboten?
 - NOT NULL
 - Löschregel für Vatertabelle (referenzierte Tabelle)
 - ON DELETE
 - { **CASCADE** | **RESTRICT** | **SET NULL** | **SET DEFAULT** | NO ACTION }
 - Änderungsregel für Vatertabelle (PS oder SK)
 - ON UPDATE
 - { **CASCADE** | **RESTRICT** | **SET NULL** | **SET DEFAULT** | NO ACTION }
 - Die Option `RESTRICT` wird hier explizit aufgeführt; sie entspricht dem Fall, dass die gesamte Klausel weggelassen wird.

Referentielle Aktionen

Spezifikation

■ **RESTRICT**

Operation wird nur ausgeführt, wenn keine zugehörigen Sätze (FS-Werte) vorhanden sind

■ **CASCADE**

Operation propagiert zu allen zugehörigen Sätzen

■ **SET NULL**

FS wird in zugehörigen Sätzen auf `NULL` gesetzt

■ **SET DEFAULT**

FS wird in den zugehörigen Sätzen auf einen benutzerdefinierten Default-Wert gesetzt

■ **NO ACTION**

- Für die spezifizierte Referenz wird keine referentielle Aktion ausgeführt.
- Durch eine DB-Operation können jedoch mehrere Referenzen (mit unterschiedlichen Optionen) betroffen sein;
- Am Ende aller zugehörigen referentiellen Aktionen wird die Einhaltung der referentiellen Integrität geprüft

Referentielle Aktionen

Beispiel

- Klauseln **ON DELETE** und **ON UPDATE** geben an, welche referentiellen Aktionen bei einem **DELETE** bzw. **UPDATE** auf die referenzierte Vartertabelle ausgeführt werden sollen, um die referentielle Integrität zu gewährleisten

Beispiel

```
CREATE TABLE Professor(  
  PersNr      INTEGER PRIMARY KEY,  
  Name        VARCHAR(30) NOT NULL,  
  ...);
```

```
CREATE TABLE Vorlesung(  
  VorlNr      INTEGER PRIMARY KEY,  
  Titel       VARCHAR(30),  
  SWS         INTEGER,  
  gelesenVon  INTEGER,  
  CONSTRAINT FK_gelesenVon  
    FOREIGN KEY (gelesenVon)  
    REFERENCES Professor(PersNr)  
    ON DELETE SET NULL  
);
```

Referentielle Aktion wird
auf Vorlesung-Tabelle
(=Kindtabelle) ausgeführt

Kaskadierendes Löschen

- Vorsicht bei der Verwendung von `ON DELETE CASCADE`. Es kann zu kaskadierendem Löschen kommen.

Beispiel

```
CREATE TABLE Vorlesung(  
  VorlNr      INTEGER PRIMARY KEY,  
  ...  
  gelesenVon  INTEGER,  
  CONSTRAINT FK_gelesenVon  
    FOREIGN KEY (gelesenVon)  
    REFERENCES Professor(PersNr)  
    ON DELETE CASCADE  
);  
  
CREATE TABLE hoeren(  
  ...  
  VorlNr      INTEGER,  
  CONSTRAINT FK_VorlNr  
    FOREIGN KEY (VorlNr)  
    REFERENCES Vorlesung(VorlNr)  
    ON DELETE CASCADE  
);
```



Referenzielle Aktionen

Systemunterstützung

SQL-1999 FOREIGN KEY ON DELETE	Oracle	DB2	Postgres
NO ACTION	(✓)	✓	✓
RESTRICT	–	✓	✓
CASCADE	✓	✓	✓
SET NULL	✓	✓	✓
SET DEFAULT	–	–	✓
ON UPDATE			
NO ACTION	(✓)	✓	✓
RESTRICT	–	✓	✓
CASCADE	–	–	✓
SET NULL	–	–	✓
SET DEFAULT	–	–	✓

Referentielle Integritätsbedingungen

Rekursive Fremdschlüssel

- Zur Abbildung hierarchischer Strukturen
 - für jeden Wert `Vorgesetzter` existiert Wert in Spalte `Mitarbeiter_Nr`

Beispiel

```
CREATE TABLE Mitarbeiter (
  MitarbeiterNr NUMBER(8),
  Vorgesetzter NUMBER(8),
  ...,
  CONSTRAINT Mitarbeiter_PK
    PRIMARY KEY (MitarbeiterNr),
  CONSTRAINT Mitarbeiter_Vorgesetzter_FK
    FOREIGN KEY (Vorgesetzter)
      REFERENCES Mitarbeiter (MitarbeiterNr)
    ON DELETE CASCADE,
  ...
)
```

Mitarbeiter_Vorgesetzter_FK

Mitarbeiter



<u>MitarbeiterNr</u>	<u>Vorgesetzter</u>	...
....		...

Überblick

- Einführung Datendefinitionssprachen
- Datendefinition in SQL
 - Schemata
 - Datentypen
 - Attribute und Tabellen
 - Referentielle Integrität
 - Schemaevolution
- Anhang I: Universitätsschema
- Anhang II: Definitionen

Schemaevolution 1/5

- Wachsender oder sich ändernder Informationsbedarf
 - Erzeugen/Löschen von Tabellen
 - Hinzufügen, Ändern und Löschen von Spalten
 - Anlegen/Ändern von referentiellen Beziehungen
 - Hinzufügen, Modifikation, Wegfall von Integritätsbedingungen

Schemaevolution 2/5

Verwalten der Tabellenstruktur

- Bei Tabellen können dynamisch (während ihrer Lebenszeit) Schemaänderungen durchgeführt werden

Schemaänderungsanweisungen:

```
ALTER TABLE base-table
{ ADD [COLUMN] column-def
| ALTER [COLUMN] column
  { SET default-def | DROP DEFAULT }
| DROP [COLUMN] column { RESTRICT | CASCADE }
| ADD base-table-constraint-def
| DROP CONSTRAINT constraint { RESTRICT | CASCADE } }
```

Schemaevolution 3/5

Ändern der Tabellenstruktur - Spalten

■ Mit der **ALTER TABLE**-Anweisung können u.a.

- neue Spalten hinzufügen

```
ALTER TABLE Professor ADD (Titel VARCHAR(30));
```

- vorhandene Spalten ändern

```
ALTER TABLE Professoren MODIFY (Titel VARCHAR(40));
```

- Spalten löschen

```
ALTER TABLE Professor DROP COLUMN Titel;
```

```
ALTER TABLE Professor DROP COLUMN Titel RESTRICT;
```

- Wenn die Spalte die einzige der Tabelle ist, wird die Operation zurückgewiesen
- Da **RESTRICT** spezifiziert ist, wird die Operation zurückgewiesen, wenn die Spalte in einer **Sicht** oder einer **Integritätsbedingung** (z.B. Check-Constraint) referenziert wird.
- **CASCADE** dagegen erzwingt die **Folgelöschung** aller **Sichten** und **Integritätsbedingungen**, die von der Spalte abhängen

Schemaevolution 4/5

Löschen der Tabellenstruktur

■ Alle Daten und die Tabellenstruktur löschen

```
DROP TABLE Professor;  
DROP TABLE Professor CASCADE;
```

- Mit der **CASCADE-Option** werden 'abhängige' Objekte (z.B. referentielle Integritätsbedingungen) ebenfalls gelöscht.

ACHTUNG bei Oracle:

```
DROP TABLE Professor CASCADE CONSTRAINT;
```

- **RESTRICT** verhindert Löschen, wenn die zu löschende Tabelle noch durch Sichten oder Integritätsbedingungen referenziert wird.

Schemaevolution 5/5

Verwalten der Integritätsbedingungen (Constraints)

■ Constraint wird im Zuge der Tabellendefinition

- angelegt und aktiviert

```
CREATE TABLE Professor (  
    ...,  
    Rang CHAR(2) CONSTRAINT Check_Rang CHECK(Rang IN ('C2','C3'));
```

■ Constraint wird zu bestehender Tabelle

- hinzugefügt und sofort aktiviert

```
ALTER TABLE Professor  
    ADD CONSTRAINT Check_Rang CHECK(Rang IN ('C2','C3'));
```

- hinzugefügt und deaktiviert

```
ALTER TABLE Professor  
    ADD CONSTRAINT Check_Rang CHECK(Rang IN ('C2','C3')) DISABLE;
```

■ Constraints wird deaktiviert

```
ALTER TABLE Professor MODIFY CONSTRAINT Check_Rang DISABLE;
```

■ Bestehende (deaktivierte) Constraints werden aktiviert

```
ALTER TABLE Professor MODIFY CONSTRAINT Check_Rang ENABLE;
```

■ Constraints wird gelöscht

```
ALTER TABLE Professor DROP CONSTRAINT Check_Rang;
```

Zusammenfassung

Datendefinition mit SQL

- Datendefinition
 - CHECK-Bedingungen für Wertebereiche, Attribute und Tabellen
- Kontrolle der Beziehungen
 - Referentielle Integrität von FS → PS/SK wird stets gewährleistet
 - Rolle PRIMARY KEY, UNIQUE, NOT NULL
 - Es kann nicht spezifiziert werden, dass ein Tupel in der Vartabelle einen Tupel in der Kindtabelle haben muss
- Wartung der referentiellen Integrität
 - Optionen für referentielle Aktionen
 - Es sind stets sichere Schemata anzustreben
- Schemaevolution
 - Änderung/Erweiterung von Spalten, Tabellen, Integritätsbedingungen
- **Integritätsbedingungen überlegt einsetzen!**
 - Jede Integritätsbedingung muss bei Veränderungen am Datenbestand überprüft werden und kostet somit Rechenzeit.

Anhang I: Universitätsschema

Universitätsschema

```
CREATE TABLE Student
  ( MatrNr      NUMBER(8,0) NOT NULL,
    Name        VARCHAR(30) NOT NULL,
    Semester    NUMBER(2,0) DEFAULT 0,
    CONSTRAINT pk_student PRIMARY KEY (MatrNr));

CREATE TABLE Professor
  ( PersNr      NUMBER(6,0) NOT NULL,
    Name        VARCHAR(30) NOT NULL,
    Rang        CHAR(2) NOT NULL,
    Raum        NUMBER(4,0) UNIQUE,
    CONSTRAINT pk_professor PRIMARY KEY (PersNr),
    CONSTRAINT check_professor_rang CHECK (Rang IN ('C2', 'C3', 'C4')));

CREATE TABLE Assistent
  ( PersNr      NUMBER(6,0) NOT NULL,
    Name        VARCHAR(30) NOT NULL,
    Fachgebiet   VARCHAR(50) DEFAULT 'Ist noch zu definieren!',
    Boss        NUMBER(6,0) NOT NULL,
    CONSTRAINT pk_assistent PRIMARY KEY (PersNr),
    CONSTRAINT fk_assistent_professor FOREIGN KEY (Boss)
    REFERENCES Professor (PersNr));
```

Anhang I: Universitätsschema

Universitätsschema

```
CREATE TABLE Vorlesung
( VorlNr          NUMBER(5,0) NOT NULL,
  Titel           VARCHAR(30) NOT NULL,
  SWS             NUMBER(2,0) NOT NULL,
  gelesenVon      NUMBER(6,0) ,
  CONSTRAINT     pk_vorlesung PRIMARY KEY (VorlNr) ,
  CONSTRAINT     fk_vorlesung_professor FOREIGN KEY (gelesenVon)
                        REFERENCES Professor (PersNr)) ;

CREATE TABLE hoeren
( MatrNr          NUMBER(8,0) NOT NULL,
  VorlNr          NUMBER(5,0) NOT NULL,
  CONSTRAINT     pk_hoeren PRIMARY KEY (MatrNr, VorlNr) ,
  CONSTRAINT     fk_hoeren_student FOREIGN KEY (MatrNr)
                        REFERENCES Student (MatrNr) ON DELETE CASCADE ,
  CONSTRAINT     fk_hoeren_vorlesung FOREIGN KEY (VorlNr)
                        REFERENCES Vorlesung (VorlNr) ON DELETE CASCADE) ;
```

Anhang I: Universitätsschema

Universitätsschema

```
CREATE TABLE voraussetzen
  ( Vorgaenger      NUMBER(5,0) NOT NULL,
    Nachfolger      NUMBER(5,0) NOT NULL,
    CONSTRAINT pk_voraussetzen PRIMARY KEY (Vorgaenger, Nachfolger),
    CONSTRAINT fk_voraussetzen_vorlesung_vor FOREIGN KEY (Vorgaenger)
      REFERENCES Vorlesung (VorlNr) ON DELETE CASCADE,
    CONSTRAINT fk_voraussetzen_vorlesung_nach FOREIGN KEY (Nachfolger)
      REFERENCES Vorlesung (VorlNr) ON DELETE CASCADE);

CREATE TABLE pruefen
  ( MatrNr          NUMBER(8,0) NOT NULL,
    VorlNr          NUMBER(5,0) NOT NULL,
    PersNr          NUMBER(6,0) NOT NULL,
    Note            NUMBER(2,1) NOT NULL,
    CONSTRAINT pk_pruefen PRIMARY KEY (MatrNr, VorlNr),
    CONSTRAINT fk_pruefen_student FOREIGN KEY (MatrNr)
      REFERENCES Student (MatrNr) ON DELETE CASCADE,
    CONSTRAINT fk_pruefen_vorlesung FOREIGN KEY (VorlNr)
      REFERENCES Vorlesung (VorlNr),
    CONSTRAINT fk_pruefen_professor FOREIGN KEY (PersNr)
      REFERENCES Professor (PersNr),
    CONSTRAINT check_pruefen_note CHECK (Note BETWEEN 1.0 AND 5.0));
```

Anhang II: Definitionen

Referentielle Integrität

Fremdschlüssel müssen entweder auf existierende Tupel einer anderen Relation verweisen oder einen NULL-Wert enthalten.

Anhang III: Exkurs CHECK-Klausel

■ CHECK-Klausel

- Bedingungen können beliebig komplex sein, auch eine Anfrage enthalten
 - Achtung: nur sehr rudimentär implementiert!

Beispiele: Zeilenbedingungen

```
CREATE TABLE Student (
  MatrNr      INTEGER PRIMARY KEY,
  Name        VARCHAR (30) NOT NULL,
  Semester    INTEGER NOT NULL,
  CONSTRAINT Check_Student CHECK (Semester BETWEEN 1 AND 13 AND MatrNr > 0))
```

Beispiele: Tabellenbedingungen

```
CREATE TABLE Student (
  ... ,
  CONSTRAINT Check_StudentAnzahl CHECK (1500 > (SELECT COUNT (*) FROM Student)))
```

SQL-1999	Oracle	DB2	Postgres	MySQL
CHECK (Spaltenbedingung)	✓	✓	✓	✓
CHECK (Zeilenbedingung)	✓	✓	✓	✓
CHECK (Tabellenbedingung)	-	-	-	-
CHECK (Datenbankbedingung)	-	-	-	-